

code

User Churn Rate

Data analysis using SQL

Alexander K.

2025-09-27

Table of Contents

1. Overview
2. Familiarity with data
3. Calculate churn rate

1. Overview

1. Overview

Codeflix, a streaming video startup, is interested in measuring their **user churn rate**. In this project, we'll be helping them answer these questions about their churn.

2. Familiarity with data

2. Familiarity with data

The dataset contains 4 columns:

- id - the subscription id
- subscription_start - the start date of the subscription
- subscription_end - the end date of the subscription
- segment - this identifies which segment the subscription owner belongs to (there are segments of subscriptions: 87 and 30)

The range of months of data provided is 4 months: from dec 2016 to march 2017.

id	subscription_start	subscription_end	segment
1	2016-12-01	2017-02-01	87
2	2016-12-01	2017-01-24	87
3	2016-12-01	2017-03-07	87
4	2016-12-01	2017-02-12	87

```
-- #1 Take a look at the first few rows
```

```
SELECT
  *
FROM
  subscriptions
LIMIT
  15;
```

```
-- #2 Determine the range of months of data provided
```

```
SELECT
  STRFTIME ('%m', subscription_start) AS 'month',
  COUNT(*)
FROM
  subscriptions
GROUP by
  1;
```

3. Calculate churn rate

3.1. Using WITH and CROSS JOIN

Creating temporary tables by WITH statements.

CROSS JOIN temporary months table with main data 'subscriptions'.

id	subscription_start	subscription_end	segment	first_day	last_day
1	2016-12-01	2017-02-01	87	2017-01-01	2017-01-31
1	2016-12-01	2017-02-01	87	2017-02-01	2017-02-28
1	2016-12-01	2017-02-01	87	2017-03-01	2017-03-31
2	2016-12-01	2017-01-24	87	2017-01-01	2017-01-31

```
- #4 Create a temporary table, cross_join, from
subscriptions and months
```

```
WITH
  months AS (
    SELECT
      '2017-01-01' AS first_day,
      '2017-01-31' AS last_day
    UNION
    SELECT
      '2017-02-01' AS first_day,
      '2017-02-28' AS last_day
    UNION
    SELECT
      '2017-03-01' AS first_day,
      '2017-03-31' AS last_day
  ),
  cross_join AS (
    SELECT
      *
    FROM
      subscriptions
      CROSS JOIN months
  )
SELECT
  *
FROM
  cross_join
LIMIT
  50;
```

3.2. Creating temporary table - status

Create a temporary table, status, from the cross_join table.

This table contains month column and two statues for each segments:

- is_active (1 - users who existed before the beginning of the month; 0 - users who did not exist)
- is_canceled (1 - if the subscription is canceled during the month; 0 - otherwise)

id	month	is_active_87	is_active_30	is_canceled_87	is_canceled_30
1	1	1	0	0	0
1	2	0	0	1	0
1	3	0	0	0	0
2	1	1	0	1	0

```
...
-- status temp table
status AS (
  SELECT
    id,
    STRFTIME ('%Y_%m', first_day) AS 'Year_month',
    segment,
    -- create is_active column
    CASE
      WHEN (subscription_start < first_day)
      AND (
        subscription_end > first_day
        OR subscription_end IS NULL
      ) THEN 1
      ELSE 0
    END AS 'is_active',
    -- create is_canceled column
    CASE
      WHEN (subscription_end BETWEEN first_day AND
last_day) THEN 1
      ELSE 0
    END AS 'is_canceled'
  FROM
    cross_join
),
...
```

3.3. Calculate the churn rates

Creating a `status\_aggregate` temporary table that is a SUM of the active and canceled subscriptions for each segment.

Calculating the churn rates for two (or maybe more) segments, using `status\_aggregate` table and GROUP BY statement.

Year_month	segment	sum_active	sum_canceled	churn_rate
2017_02	30	518	38	0.0734
2017_01	30	291	22	0.0756
2017_03	30	716	84	0.1173
2017_01	87	278	70	0.2518
2017_02	87	462	148	0.3203
2017_03	87	531	258	0.4859

```
-- status_aggregate temp table
status_aggregate AS (
  SELECT
    Year_month,
    segment,
    SUM(is_active) AS 'sum_active',
    SUM(is_canceled) AS 'sum_canceled',
    ROUND(1.0 * SUM(is_canceled) / SUM(is_active), 4) AS
      'churn_rate'
  FROM
    status
  GROUP BY
    1,
    2
)
```